

CAN

CAN

Bussen, framen och kontrollern

Erik Pihl

12 september 2026

CAN - bussen, ramen och kontrollern

En kort bok om CAN, skriven för den som ska bygga en CAN-nod: antingen kontrollern i hårdvara eller drivrutinen i mjukvara.

Boken beskriver CAN som protokoll: bussen, ramen, bitstoppningen, checksumman, arbitreringen och bittajmingen. Sista kapitlet beskriver den förenklade kontrollern som byggs och drivs i undervisningen, vad den implementerar av allt det föregående, och vad den medvetet låter bli.

Brödtexten är på svenska. All kod, alla registernamn och alla signalnamn är på engelska, precis som i koden och i de gemensamma specifikationerna.

Text och figurer: Erik Pihl.

Bokens text och figurer, liksom kursmaterialet den är satt från, är licensierade under Creative Commons Erkännande 4.0 Internationell (CC BY 4.0). Du får kopiera, sprida och bearbeta materialet i vilket syfte som helst, även kommersiellt, förutsatt att du anger upphovsperson och licens. Licensen finns på <https://creativecommons.org/licenses/by/4.0/deed.sv>.

Satt med Lua[®]TeX i TeX Gyre Pagella och DejaVu Sans Mono.

Den här utgåvan: 12 september 2026.

Innehåll

Förord	vii
1 Bussen	1
1.1 Problemet	1
1.2 Vad en delad buss kostar	2
1.3 Dominant och recessiv	2
1.4 Ledningarna	3
1.5 Hastighet och längd	4
2 Framen	7
2.1 Framens form	7
2.2 Identifieraren	8
2.3 RTR och fjärrframes	9
2.4 Kontrollfältet och DLC	10
2.5 CRC-fältet	10
2.6 ACK-fältet	10
2.7 EOF och intermission	11
3 Bitstoppning och CRC-15	13
3.1 Problemet med lika bitar	13
3.2 Femregeln	13
3.3 CRC-15	14
4 Arbitrering	19
4.1 Ingen mästare	19
4.2 Två noder, bit för bit	20
4.3 Tre noder	20
4.4 Vad förloraren gör	21
4.5 Svält och prioritetsinversion	21
4.6 Vad sändaren inte får veta	22

5	Bittajming	23
5.1	Problemet	23
5.2	Bitperioden i segment	23
5.3	Synkronisering	24
5.4	Att räkna ut tajmingen	25
5.5	Vad som händer när det går fel	25
6	Felhantering	27
6.1	Fem sorters fel	27
6.2	Felramen	28
6.3	Felräknarna	28
6.4	Vad mjukvaran ser	29
7	Den förenklade kontrollern	33
7.1	Blocken	34
7.2	Vad som finns med	34
7.3	Vad som inte finns med	35
7.4	Varför ett registerlager behövs	35
7.5	De tre begränsningarna som når ända upp	36
7.6	Vad en riktig kontroller skulle lägga till	37
A	Svar på övningarna	39
A.1	Bussen	39
A.2	Framen	40
A.3	Bitstoppning och CRC-15	41
A.4	Arbitrering	42
A.5	Bittajming	44
A.6	Felhantering	45
A.7	Den förenklade kontrollern	46

Förord

Ett hemsnickrat protokoll är lärorikt att bygga. Man bestämmer själv var framen börjar, hur lång den är, hur den skyddas mot fel, och vad som ska hända när ett svar uteblir. Varje mekanism blir ens egen — och varje mekanism blir något som måste skrivas och underhållas.

Den här boken handlar om vad som händer när någon annan redan har bestämt allt det, en gång, och gjutit det i kisel.

CAN är från 1986, det sitter i varje bil som byggts sedan mitten av nittioalet, och det löser nästan exakt de problem ni just löste själva — fast i hårdvara, och bättre. Framing, checksumma, kollisionshantering och kvittens ligger i kontrollern. Det som återstår för mjukvaran är att lämna över en identifierare, en längd och några databytes, och sedan läsa av hur det gick.

Observera: Att CAN gör det åt er betyder inte att ni slipper förstå det. Bygger ni kontrollern är protokollet er kravspecifikation, bit för bit. Skriver ni drivrutinen måste ni hantera precis det kontrollern *inte* kan säga er: varför en förlorad arbitring ser ut som en lyckad sändning, varför en frame som ingen kvitterar ändå fullbordas, och varför en mottagen frame kan försvinna under näsan på er. Allt det är konsekvenser av hur CAN fungerar, och de står i den här boken.

Vad boken täcker

Kapitel 1 börjar vid ledningarna: varför en delad buss, vad dominant och recessiva bitar är, och varför den asymmetrin är hela grunden. Kapitel 2 går igenom CAN-framen fält för fält. Kapitel 3 tar bitstopningen och CRC-15:an, de två mekanismerna som gör att en frame överlever en störig buss. Kapitel 4 arbetar igenom arbitringen bit för bit, för hand. Kapitel 5 handlar om bittajming och sampelpunkten: hur noder utan gemensam klocka ändå läser samma bit samtidigt. Kapitel 6 beskriver felhanteringen i riktig CAN — felframes, felräknare och bus-off.

Kapitel 7 är bokens enda kapitel som inte handlar om CAN i allmänhet. Det beskriver den förenklade kontrollern som byggs i undervisningen, vad den implementerar av allt det föregående, och vad den medvetet låter bli. Det kapitlet är det som kommer till användning vid felsökning — från båda hållen.

Vad boken inte täcker

Boken beskriver **klassisk CAN** med 11-bitars identifierare, alltså CAN 2.0A. Utökade 29-bitars identifierare (2.0B) nämns där de påverkar framens form, men arbetas inte igenom. CAN FD och CAN XL ligger utanför, liksom högre lager som CANopen och J1939.

Boken beskriver inte heller elektriken i detalj: differentiell signalering, termineringens fysik och kabellängdens inverkan berörs bara så långt det behövs för att förstå varför bussen ser ut som den gör.

Övningar

Varje kapitel slutar med några övningar. De görs med papper och penna — det finns ingen kod att kompilera i den här boken — och svaren på samtliga står i Appendix A.

Övningarna är korta med avsikt. De är till för att kontrollera att kapitlet fastnade, inte för att ta en kväll.

Bussen

Två ledningar, många noder, och en asymmetri som avgör allt.

I DET HÄR KAPITLET

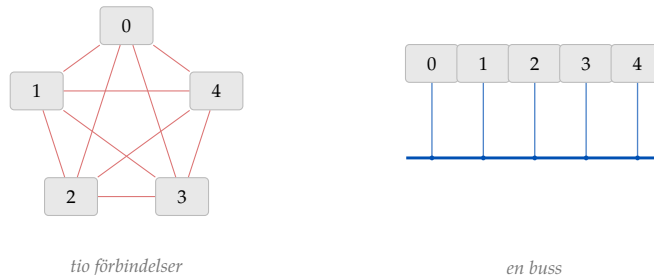
- Varför punkt-till-punkt slutar fungera när ett system växer.
- Den delade bussen, och vad den kostar.
- Dominanta och recessiva bitar, och den logiska OCH-funktionen bussen utför.
- Differentiell signalering och terminering, så långt den som bygger en nod behöver det.

1.1 Problemet

Ett inbyggt system med fem enheter som alla behöver prata med varandra behöver tio förbindelser om varje par kopplas ihop direkt. Med tio enheter är det fyrtiofem. Formeln är $n(n - 1)/2$, och den är obeveklig: kablaget växer kvadratisk medan nyttan växer linjärt.

I en bil är det inte en teoretisk invändning. Motorstyrning, ABS, instrumentpanel, växellåda, krockkuddar, klimatanläggning — och varje förbindelse är koppar som väger, kostar, tar plats och kan gå av.

Lösningen är att sluta koppla ihop enheter, och i stället koppla in dem. En enda buss, som alla noder delar:



Figur 1.1 Punkt-till-punkt mot delad buss. Fem noder kräver tio förbindelser i det vänstra fallet och en buss i det högra.

Det som var $n(n-1)/2$ förbindelser blir n anslutningar till en gemensam ledning. Priset är att bussen nu är en delad resurs, och delade resurser måste fördelas.

1.2 Vad en delad buss kostar

Tre problem uppstår i samma ögonblick som två noder delar en ledning, och CAN:s hela konstruktion är svaret på dem:

Problem	CAN:s svar
Alla hör allt, så en mottagare måste veta vad som angår den	Identifieraren beskriver <i>innehållet</i> , och varje nod filtrerar själv (kapitel 2)
Två noder kan börja sända samtidigt	Icke-förstörande arbitrerings (kapitel 4)
En störning på ledningen träffar alla	Bitstoppning och CRC-15 (kapitel 3), plus felhantering (kapitel 6)

Lägg märke till vad som *inte* står i tabellen: adressering. CAN har ingen destinationsadress. Det är inte en förenkling utan ett designval, och kapitel 2 går igenom varför.

1.3 Dominant och recessiv

Här ligger den enda idé man måste ha helt klar för sig för att resten av CAN ska bli begripligt.

En CAN-buss har två tillstånd, och de är **inte** symmetriska:

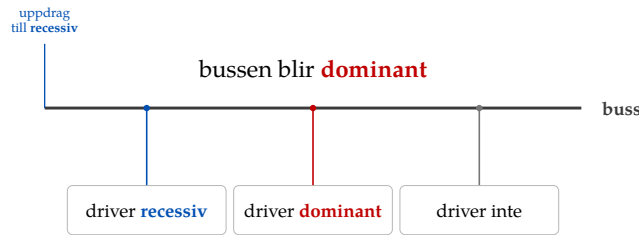
Tillstånd	Logisk nivå	Vinner?
dominant	0	Ja
recessiv	1	Nej

Regeln är enkel och får konsekvenser hela boken igenom:

Bussen är dominant om *minst en* nod driver den dominant. Den är recessiv bara om *alla* noder släpper den.

Elektriskt är det en öppen dränering med ett uppdragsmotstånd: en nod kan bara *dra ned* ledningen, aldrig driva upp den. Släpper alla, dras den upp av sig själv.

Logiskt är bussen alltså en OCH-grind över alla noders utgångar:



Figur 1.2 Bussen som wired-AND. Tre noder, och bussens nivå som funktion av vad de driver: en enda dominant nod räcker för att bussen ska bli dominant.

1.3.1 Varför asymmetrin behövs

Anta i stället en symmetrisk buss, där en etta och en nolla som möts ger ett odefinierat mellanläge. Då är en kollision en *förstörelse*: båda framesen blir obrukbara, båda noderna måste upptäcka det, vänta olika länge och försöka igen. Det är så Ethernet gjorde det, och det är därför Ethernet behövde slumpmässig backoff.

Med asymmetrin blir en kollision i stället en *jämförelse*. Noden som sänder recessivt och läser tillbaka dominant vet omedelbart två saker: att någon annan också sänder, och att den andra har företräde. Den drar sig ur, tyst, mitt i ramen — och den vinnande nodens frame fortsätter utan ett enda förlorat bitintervall.

Det kallas **icke-förstörande arbitrerings**, och det är CAN:s mest eleganta egenskap. Kapitel 4 arbetar igenom den bit för bit.

1.3.2 Att läsa tillbaka det man sänder

En förutsättning som är lätt att gå förbi: **varje sändande nod läser hela tiden av bussen medan den sänder**. Det är så den upptäcker att den förlorat arbitrerings, och det är också så den upptäcker att bussen inte gör som den blev tillsagd.

Den förenklade kontrollern gör detta under arbitreringsfältet. En fullständig CAN-kontroller gör det under *hela* ramen, och rapporterar ett *bitfel* så fort det den läser tillbaka skiljer sig från det den sände. Skillnaden står i kapitel 7.

1.4 Ledningarna

CAN är **differentiellt**: signalen ligger som en spänningsskillnad mellan två ledare, CAN_H och CAN_L, tvinnade om varandra.

Tillstånd	CAN_H	CAN_L
recessiv	2,5 V	2,5 V
dominant	3,5 V	1,5 V

I recessivt läge driver ingen, och båda ledarna vilar på samma nivå — skillnaden är noll. I dominant läge driver den sändande noden isär dem till ungefär två volt.

Poängen med differentiell signalering är störimmunitet. En störning som slår mot kabeln träffar båda ledarna ungefär lika mycket, och eftersom mottagaren bara mäter *skillnaden* tar den inte notis om den. Att ledarna är tvinnade är vad som gör att störningen verkligen träffar båda lika.

1.4.1 Terminering

En CAN-buss termineras med **120 Ω i vardera änden**, inte i mitten, och inte fler än två motstånd oavsett hur många noder som sitter på bussen.

Motstånden matchar kabelns karakteristiska impedans så att en signal som når änden absorberas i stället för att studsas tillbaka. En felterminerad buss fungerar ofta ändå på en kort kabel vid låg hastighet, och slutar fungera precis när man börjat lita på den — vilket gör felet till ett av de obehagligare.

Vid bänken: Ett snabbt sätt att kontrollera termineringen: mät resistansen över CAN_H och CAN_L med bussen strömlös. Två parallella 120 Ω ska ge ungefär 60 Ω . Får ni 120 saknas ett motstånd; får ni 40 sitter det tre.

1.5 Hastighet och längd

CAN:s bithastighet och bussens längd hänger ihop, och skälet är arbitreringen: under arbiringsfältet måste en bit hinna ut till bussens bastersta nod *och* tillbaka innan biten är slut, annars kan inte noderna vara överens om vem som vann.

Bithastighet	Ungefärlig maximal längd
1 Mbit/s	40 m
500 kbit/s	100 m
125 kbit/s	500 m

Den förenklade kontrollern kör på **1 Mbit/s**, och bussen är en kabel på labbänken. Gränsen är alltså aldrig i närheten av att bli ett problem — men sambandet är värt att känna till, eftersom det förklarar varför fordonssystem ofta kör långsammare än de skulle kunna.

Övningar

Övning 1.1 Kablage

RÄKNING

- a) Hur många förbindelser krävs för att koppla ihop åtta enheter punkt-till-punkt?
- b) Hur många anslutningar krävs om de i stället delar en buss?
- c) Vid vilket antal enheter blir punkt-till-punkt dubbelt så många förbindelser som bussens anslutningar?

Övning 1.2 Bussens nivå

RESONEMANG

Fyra noder sitter på en buss. Under ett visst bitintervall driver nod A recessivt, nod B dominant, nod C recessivt, och nod D driver inte alls.

- a) Vilken nivå har bussen?
- b) Vilka av noderna läser tillbaka något annat än det de sände?
- c) Vad drar de för slutsats av det?

Övning 1.3 Terminering

RESONEMANG

En grupp mäter $40\ \Omega$ över en strömlös buss.

- a) Vad är fel?
- b) Bussen fungerar ändå på labbänken. Varför, och när skulle den sluta göra det?

Övning 1.4 Asymmetrin

RESONEMANG

Anta en tänkt buss där recessiv i stället vann över dominant — alltså där bussen är recessiv om minst en nod driver den recessivt.

- a) Skulle arbitreringen fortfarande fungera?
- b) Vilket identifierarvärde skulle i så fall ha högst prioritet?
- c) Varför valde CAN den ordning den valde? Tänk på vad bussen gör när ingen sänder.

KAPITEL 2

Framen

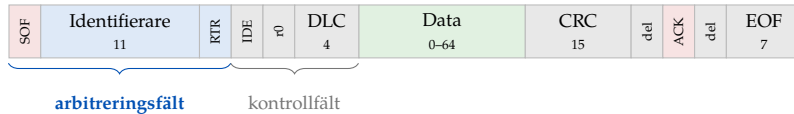
Fält för fält, från SOF till EOF.

I DET HÄR KAPITLET

- Dataframens sju fält, och vad vart och ett är till för.
- Identifieraren, som är två saker samtidigt: adress och prioritet.
- Varför CAN inte har någon destinationsadress.
- DLC, och varför den är fyra bitar när nyttolasten är högst åtta byte.
- Kvittensen som alla mottagare skriver i samma bit.

2.1 Framens form

En CAN-dataframe med 11-bitars identifierare ser ut så här:



Figur 2.1 Dataframens fält. Bredderna är i bitar, före bitstoppning.

Sju fält, och en gräns som är viktigare än de andra: allt fram till och med RTR-biten är **arbitreringsfältet**, och det är där flera samtidigt sändande noder avgör vem som får fortsätta (kapitel 4).

Fält	Bitar	Innehåll
SOF	1	Start of frame. En dominant bit, som säger att en frame börjar.
Identifierare	11	Vad ramen innehåller, och därmed dess prioritet.
RTR	1	Remote transmission request: dominant för en dataframe.
IDE	1	Identifier extension: dominant för 11-bitars identifierare.
r0	1	Reserverad, dominant.
DLC	4	Data length code: antal databytes, 0–8.
Data	0–64	Nyttolasten, noll till åtta byte.
CRC	15	Checksumma över allt föregående.
CRC-avgränsare	1	Recessiv.
ACK-lucka	1	Sändaren släpper; en mottagare drar ned.
ACK-avgränsare	1	Recessiv.
EOF	7	End of frame, sju recessiva bitar.

Efter EOF följer dessutom **intermission**, tre recessiva bitar då ingen får börja sända. Tillsammans med EOF ger det tio recessiva bitar i rad mellan två frames — ett mönster som inte kan uppstå någon annanstans, eftersom bitstoppningen (kapitel 3) gör att fem lika bitar i rad är det mesta som förekommer inuti en frame.

Räkna: $1 + 12 + 6 + 64 + 16 + 2 + 7 = 108$ bitar för den längsta dataframen, före stoppning. Bitstoppningen lägger till som mest ett tjugotal, så en maximal frame är omkring 130 bitar. Vid 1 Mbit/s tar den alltså ungefär 130 μ s. Den siffran är värd att komma ihåg: den är svaret på "hur länge kan en sändning hålla på?".

2.2 Identifieraren

Identifieraren är CAN:s mest särpräglade designval, och den är två saker samtidigt.

2.2.1 Den säger vad ramen innehåller

Den är **inte** en mottagaradress. Den beskriver *innehållet*:

Identifierare	Betydelse
0x301	Motorvarvtal
0x302	Kylvätsketemperatur
0x400	Statusförfrågan

En nod som sänder 0x301 säger inte "till instrumentpanelen", den säger "varvtalet är detta". Vem som lyssnar är inte sändarens problem, och det får konsekvenser som är lätta att missa i början:

- **Flera mottagare kan reagera på samma frame.** Både instrumentpanelen och växellådsstyrningen kan behöva varvtalet, och ingen av dem behöver veta om den andra.

- **En ny nod kan läggas till utan att någon gammal ändras.** Den börjar bara lyssna efter de identifierare den bryr sig om.
- **Filtreringen ligger hos mottagaren.** Varje nod tar emot varje frame och avgör själv om den angår den — i mjukvara, eller i kontrollerns filterhårdvara.

2.2.2 Den bestämmer prioritet

Samtidigt är identifieraren framens prioritet, och där gäller att **lägre numeriskt värde vinner**. Det följer direkt av kapitel 1: identifieraren sänds MSB först, en nolla är dominant, och den dominant biten vinner.

Det betyder att adressrymden måste planeras. En identifierare är inte bara ett namn; den är en plats i en prioritetsordning som gäller hela bussen. Bromsar får låga nummer, kupévärme höga.

En vanlig fallgrop: Två noder får aldrig sända *samma* identifierare. Två frames med samma identifierare är bit för bit identiska genom hela arbitreringsfältet, så ingen av noderna upptäcker den andra — och så fort deras datafält skiljer sig åt uppstår ett bitfel mitt i ramen. Identifierare tilldelas per sändande nod, inte per meddelandetyp.

2.2.3 Utökade identifierare

CAN 2.0B tillåter 29-bitars identifierare. Skillnaden syns i IDE-biten: recessiv i stället för dominant, och därefter följer ytterligare 18 identifierarbitar. Den här boken håller sig till 11 bitar, men det är värt att veta varför valet syns i ramen: en nod som bara förstår 11 bitar måste kunna se, tidigt och entydigt, att den inte förstår den här ramen.

Notera också prioritetskonsekvensen: en 11-bitars frame med samma inledande bitar vinner alltid över en 29-bitars, eftersom dess IDE-bit är dominant.

2.3 RTR och fjärrframes

RTR-biten skiljer två sorters frames:

RTR	Sorts frame	Betydelse
dominant (0)	Dataframe	"Här är datan."
recessiv (1)	Fjärrframe	"Skicka mig datan."

En **fjärrframe** har samma identifierare som den dataframe den efterfrågar, en DLC men *inget* datafält. Tanken är att en nod ska kunna be om ett värde i stället för att vänta på att det sänds.

I praktiken används fjärrframes sällan i moderna konstruktioner. De komplicerar mer än de tillför: två noder kan svara på samma begäran, och en nod som inte förstår mekanismen ser en frame med en DLC som inte stämmer med datafältets längd.

Att RTR ligger *i* arbitreringsfältet är däremot inte godtyckligt. Det gör att en dataframe alltid vinner över en fjärrframe med samma identifierare — den som har datan får företräde över den som ber om den, vilket är precis rätt prioritering.

2.4 Kontrollfältet och DLC

Tre bitar administration, sedan fyra bitar längd:

- **IDE** — dominant för 11 bitars identifierare.
- **r0** — reserverad, sänds dominant. Den finns för framtida bruk, och den framtiden kom aldrig för klassisk CAN.
- **DLC** — fyra bitar, värde 0–8.

Fyra bitar rymmer 0–15, men bara 0–8 är meningsfulla. Vad händer med 9–15? I standarden är de reserverade och ska behandlas som 8. I praktiken är det en av de punkter där kontrollrar skiljer sig åt — och det är därför en drivrutin som *sänder* aldrig ska producera en DLC över 8, oavsett vad hårdvaran råkar göra med den.

2.5 CRC-fältet

Femton bitars checksumma, beräknad över allt från SOF till och med datafältet, följd av en recessiv avgränsare. Hur den beräknas står i kapitel 3.

Avgränsaren finns av en praktisk anledning: den ger mottagarna en känd, recessiv bit att ta sats i innan ACK-luckan, som är den enda bit i hela framen där någon annan än sändaren driver bussen.

2.6 ACK-fältet

Här händer något som inte har någon motsvarighet i ett hemsnickrat protokoll.

Sändaren sänder ACK-luckan **recessivt** — alltså släpper den. Varje mottagare som har tagit emot framen och fått CRC:n att stämma drar samtidigt ned den **dominant**. Sändaren läser tillbaka bussen och ser:

Sändaren läser	Slutsats
dominant	Minst en nod tog emot framen felfritt
recessiv	Ingen gjorde det

Tre saker är värda att stanna vid:

Kvittensen kostar ingenting. Den är en enda bit, inuti framen, och den kräver ingen svarsframe. Jämför med ett ACK-meddelande som måste konstrueras, sändas, arbitreras om och kvitteras i sin tur.

Den säger inte vem. Alla mottagare drar ned samma bit samtidigt. Sändaren vet att *någon* hörde, aldrig hur många eller vilka. Behövs det måste applikationen svara med en egen frame.

Den säger inte att någon brydde sig. En nod som filtrerar bort framen som ointressant kvitterar den ändå, för den tog emot den felfritt. Kvittensen är ett kvitto på *överföringen*, inte på *behandlingen*.

Förenkling: Den förenklade kontrollern i kapitel 7 *släpper* ACK-luckan som sändare och *drar ned* den som mottagare, precis enligt standarden — men den läser aldrig tillbaka vad som hände där. En frame som ingen kvitterar fullbordas därför som om allt gått bra.

2.7 EOF och intermission

Sju recessiva bitar avslutar framen, och tre till av intermission håller bussen ledig innan någon får börja om.

Sju recessiva i rad är fler än bitstoppningen någonsin tillåter inuti en frame, vilket gör EOF till ett entydigt slutmärke: en mottagare som ser sju recessiva bitar vet att framen är slut, oavsett var den tappade tråden.

Det är också här felsignalering sker: en nod som upptäcker ett fel avbryter genom att sända sex dominanta bitar i rad — ett mönster som är omöjligt i en korrekt frame, och som därför inte kan förväxlas med data. Kapitel 6 går igenom det.

Övningar

Övning 2.1 Läs framen

AVKODNING

En frame har identifieraren 0x2A7, DLC 5 och databytesen 01 02 03 04 05.

- Hur många bitar är framen, före bitstoppning?
- Hur lång tid tar den vid 1 Mbit/s, före stoppning?
- Vilken identifierare skulle ha vunnit över den i en arbitrerings: 0x2A6 eller 0x2A8?

Övning 2.2 Ingen adress

RESONEMANG

Ett system har en temperatursensor och två noder som båda behöver temperaturen.

- Hur många frames måste sensorn sända?
- Hur skulle samma sak sett ut i ett protokoll med destinationsadress?
- En tredje nod tillkommer som också vill ha temperaturen. Vad måste ändras i sensorn?

Övning 2.3 Kvittensen

RESONEMANG

En nod sänder en frame och läser tillbaka en dominant ACK-lucka.

- a) Vad vet den säkert?
- b) Vad vet den inte?
- c) Noden är ensam på bussen och sänder. Vad läser den i ACK-luckan, och vad ska en standardenlig kontroller göra då?

Övning 2.4 DLC

RESONEMANG

DLC är fyra bitar men bara värdena 0–8 är meningsfulla.

- a) Varför gjordes fältet fyra bitar brett?
- b) En drivrutin skriver DLC 9 till sin kontroller. Vems ansvar är det att det inte händer, och varför just den sidan?

Bitstoppning och CRC-15

De två mekanismerna som gör att en frame överlever en störig buss.

I DET HÄR KAPITLET

- Varför en lång rad lika bitar är ett problem, och femregeln som löser det.
- Vilka fält som stoppas, och vilka som inte gör det.
- CRC-15: polynomet, den bitseriella motorn, och ett exempel räknat för hand.
- Varför samma motor både beräknar och kontrollerar.

3.1 Problemet med lika bitar

CAN-noder har ingen gemensam klocka. Varje nod har sin egen oscillator, och de går aldrig exakt lika fort. En mottagare håller takten genom att titta på *flankerna* i bitströmmen: varje gång signalen byter nivå vet den var en bitgräns låg, och kan justera sin egen räknare (kapitel 5).

Kommer det inga flanker driver mottagaren i väg. Efter tillräckligt många lika bitar i rad samplar den fel bit, och framen är förlorad.

Lösningen är att göra flankerna obligatoriska. Efter fem lika bitar i rad sätter sändaren in en **stoppbit** med motsatt värde, som mottagaren känner igen och kastar.

3.2 Femregeln

Regeln är kort: **efter fem lika bitar i rad sänds en bit med motsatt värde**, oavsett vad som kommer näst.

Ett exempel, med den insatta biten markerad:

```
in:  1 1 1 1 1 0 0 0 0 0 1
ut:  1 1 1 1 1 0 0 0 0 0 1 0 1
      ^               ^
      stoppbit       stoppbit
```

Det finns två detaljer här som är lätta att få fel, och båda är klassiska buggar:

Stoppbiten räknas själv. Efter att en stoppbit satts in börjar räkningen om med stoppbiten som första bit i den nya serien. Den är en helt vanlig bit i strömmen, för alla utom den som ska plocka bort den igen.

Regeln gäller strömmen, inte fältet. Fem lika bitar som sträcker sig över gränsen mellan identifieraren och DLC:n utlöser en stoppbit precis som fem inom ett fält. En implementation som nollställer räknaren vid varje fältbyte fungerar nästan alltid, och går sönder precis när två fält råkar sluta och börja med samma bitvärde.

3.2.1 Vilka fält som stoppas

Stoppning gäller från SOF till och med CRC-fältet. Därefter **slutar** den:

Fält	Stoppas?
SOF, arbitrering, kontroll, data, CRC	Ja
CRC-avgränsare, ACK, EOF, intermission	Nej

Att slutet inte stoppas är vad som gör EOF användbart som slutmärke. Sju recessiva bitar i rad kan inte förekomma i den stoppade delen — där är fem lika det mesta som får passera — så en mottagare som ser sju recessiva vet säkert att framen är slut.

3.2.2 Stoppfel

Mottagaren räknar likadant som sändaren. Efter fem lika bitar *måste* nästa bit vara den motsatta. Är den inte det är något fel, och mottagaren rapporterar ett **stoppfel**.

Det är ett av CAN:s billigaste feldetekteringsmedel: det kostar ingen extra information alls, bara en räknare i vardera änden.

Kostnaden: Stoppningen gör framen längre, och hur mycket beror på datan. En frame vars databitar växlar hela tiden får inga stoppbitar alls; en frame full av nollor får en var femte bit. För den längsta dataframen är påslaget som mest ett tjugotal bitar. Det är också därför en CAN-frames längd inte är exakt förutsägbar från DLC:n — vilket i sin tur är därför den som räknar på bussbelastning räknar med värstafallet.

3.3 CRC-15

Checksumman är femton bitar, beräknad över allt från SOF till och med datafältet — alltså de *verkliga* bitarna, utan stoppbitar. Stoppbitar är transportens sak och ingår inte i det som skyddas.

3.3.1 Polynomet

CAN använder

$$x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1$$

som i binär form, koefficienterna för x^{14} ned till x^0 , blir

```
100010110011001
```

Polynomet är valt för att upptäcka alla enkel-, dubbel- och trippelfel, alla felburstar upp till 15 bitar, och de allra flesta längre. Det är inte en slumpmässig konstant.

3.3.2 Motorn

Beräkningen är ett femtonbitars skiftregister med återkoppling, och den tar en bit i taget:

```
// One bit into the CRC-15 engine. crc is 15 bits wide.
const bool feedback{static_cast<bool>((crc >> 14U) & 1U) ^ bit};
crc = static_cast<std::uint16_t>((crc << 1U) & 0x7FFFU);
if (feedback)
{
    crc ^= 0x4599U; // 100010110011001
}
```

Tre rader: XOR:a den inkommande biten med registrets översta bit, skifta registret ett steg, och XOR:a in polynomet om resultatet av den första operationen var ett.

3.3.3 Ett exempel för hand

Sekvensen 10110010, matad in bit för bit i ett nollställt register:

Bit	Värde	Registret efter	Hex
1	1	100010110011001	0x4599
2	0	100111010101011	0x4EAB
3	1	001110101010110	0x1D56
4	1	111111100110101	0x7F35
5	0	0111011111110011	0x3BF3
6	0	111011111100110	0x77E6
7	1	110111111001100	0x6FCC
8	0	001101000000001	0x1A01

Checksumman är alltså 0x1A01.

3.3.4 Samma motor kontrollerar

Här ligger elegansen. Mottagaren har ingen separat kontrollfunktion: den matar in *samma* bitar plus *den mottagna checksumman* genom samma motor.

Om allt stämmer står registret på **noll** när sista CRC-biten passerat. Matar vi in 10110010 följt av 001101000000001 ur exemplet ovan blir slutvärdet 0x0000.

Det är en egenskap hos konstruktionen, inte ett specialfall: att lägga resten sist i dividenden gör den jämnt delbar. Sändare och mottagare kan därför använda exakt samma modul, utan lägesomkoppling — vilket är precis vad en hårdvarukonstruktion vill ha.

En detalj som bits: Motorn behöver ändå kunna nollställas. En avbruten frame lämnar skräp i registret, och nästa frame som börjar med ett smutsigt register får fel checksumma från första biten. Nollställningen sker vid varje framestart.

3.3.5 Vad CRC:n inte skyddar

CRC:n täcker SOF till och med datafältet. Den täcker alltså *inte* CRC-avgränsaren, ACK-fältet eller EOF. Fel i dem upptäcks i stället av formatreglerna: en avgränsare som inte är recessiv är ett formatfel (kapitel 6).

Övningar

Övning 3.1 Stoppa en ström

RÄKNING

Stoppa följande bitströmmar enligt femregeln, och markera de insatta bitarna.

- a) 0000011111
- b) 1111111111
- c) 0101010101

Hur många bitar blev var och en efter stoppning?

Övning 3.2 Avstoppa

RÄKNING

En mottagare tar emot 111110000010.

- a) Vilka bitar är stoppbitar?
- b) Vilken var den ursprungliga bitströmmen?
- c) Antag i stället att mottagaren fått 111111... Vad rapporterar den, och efter vilken bit?

Övning 3.3 CRC-motorn

RÄKNING

Mata in 1101 i ett nollställt CRC-15-register, ett steg i taget, och skriv upp registrets värde efter varje bit. Använd tabellen i avsnitt 3.3 som mall.

Övning 3.4 Varför noll?

RESONEMANG

- a) Varför står mottagarens register på noll när både data och checksumma matats in?
- b) Vilken praktisk fördel har det för en hårdvarukonstruktion?
- c) En nod matar av misstag in *stoppbitarna* i CRC-motorn också. Vad händer, och vem upptäcker det?

Arbitrering

Hur en buss utan mästare avgör vem som får sända.

I DET HÄR KAPITLET

- Multimaster: ingen nod fördelar bussen, och ingen behöver göra det.
- Arbitreringen bit för bit, arbetad för hand med två och tre noder.
- Varför den kallas icke-förstörande, och vad det sparar.
- Prioritetsinversion, svält, och hur ett verkligt system undviker dem.

4.1 Ingen mästare

De flesta bussar löser åtkomsten med en mästare. En nod bestämmer vem som får prata, och de andra väntar på tur. Det fungerar, och det kostar en nod som alla är beroende av.

CAN har ingen sådan. Varje nod får börja sända så fort bussen varit ledig i tre bitar (intermission). Börjar två noder samtidigt löser bussen själv upp det, medan det händer, utan att någon förlorar tid.

Mekanismen är den asymmetri kapitel 1 lade fram, använd på ett enda ställe: under arbitreringsfältet jämför varje sändande nod, bit för bit, det den sänder med det som faktiskt ligger på bussen.

Regeln, i sin helhet: sänder en nod **recessiv** och läser tillbaka **dominant**, har den förlorat. Den slutar sända omedelbart och blir mottagare.

Sänder den dominant och läser dominant är allt som det ska — antingen är den ensam, eller så sänder någon annan samma bit. Sänder den recessivt och läser recessivt likaså.

Den enda kombinationen som säger något är alltså recessiv ut, dominant in: någon annan sänder, och den andra har en nolla där vi har en etta. Eftersom identifieraren sänds MSB först betyder det att den andras identifierare är numeriskt *lägre*, och därmed har högre prioritet.

4.2 Två noder, bit för bit

Nod A vill sända identifieraren 0x2A7, nod B 0x2A3. Båda börjar samtidigt.

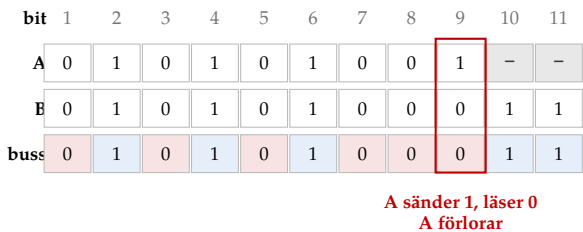
Binärt, elva bitar, MSB först:

```
bit:          1 2 3 4 5 6 7 8 9 10 11
A (0x2A7):    0 1 0 1 0 1 0 0 1  1  1
B (0x2A3):    0 1 0 1 0 1 0 0 0  1  1
                ^ här skiljer de sig
```

Bit	A sänder	B sänder	Bussen	Händelse
1–8	samma	samma	samma	Ingen märker något
9	recessiv (1)	dominant (0)	dominant	A läser tillbaka dominant: A förlorar
10–11	—	sänder vidare	B:s bitar	B fortsätter ostört

Notera vad som *inte* händer: ingen kollision, ingen förlorad tid, ingen omstart. B:s frame fortsätter från bit 9 som om A aldrig hade funnits. A:s frame sändes aldrig — den bara slutade, mitt i, och A väntar ut B:s frame för att försöka igen efteråt.

Därför heter det **icke-förstörande arbitrering**: konflikten löses utan att någons data förstörs.



Figur 4.1 Arbitrering mellan två noder. A drar sig ur och blir mottagare vid bit 9; B:s frame löper vidare oavbrutet.

4.3 Tre noder

Med fler noder är det samma regel, tillämpad samtidigt av alla. Tre noder med identifierarna 0x100, 0x0FF och 0x2A7:

```
bit:          1 2 3 4 5 6 7 8 9 10 11
A (0x100):    0 0 1 0 0 0 0 0 0  0  0
B (0x0FF):    0 0 0 1 1 1 1 1 1  1  1
C (0x2A7):    0 1 0 1 0 1 0 0 1  1  1
                ^ ^
```

```
| bit 3: A sänder recessivt, B dominant -> A faller bort
bit 2: C sänder recessivt, A och B dominant -> C faller bort
```

Vid bit 1 sänder alla tre dominant, så ingen märker något. Vid bit 2 faller C bort, vid bit 3 faller A bort. B vinner, och har lägst identifierare — vilket alltid är utfallet, eftersom "lägst numeriskt" och "först dominant bit" är samma sak när bitarna sänds MSB först.

4.4 Vad förloraren gör

Den förlorande noden blir mottagare för resten av framen. Det är inte bara artighet: den *ska* ta emot den vinnande framen, eftersom den mycket väl kan vara adresserad till den.

När bussen blivit ledig igen försöker den om. Och eftersom dess identifierare är oförändrad kommer den att förlora igen om samma högprioriterade nod sänder igen — vilket leder till nästa avsnitt.

Förenkling: Den förenklade kontrollern i kapitel 7 tar *inte* emot framen den förlorade mot. Den lämnar sändarrollen, sätter sin felflagga och går tillbaka till vila. Det är en medveten förenkling, och den är synlig för mjukvaran: en nod som förlorat en arbitrerings missar den frame som vann.

4.5 Svält och prioritetsinversion

Arbitreringen är rättvis mot prioriteten, inte mot noderna. En nod med hög identifierare på en hårt belastad buss kan i princip aldrig komma fram — den förlorar varje gång.

Det är ett känt och accepterat beteende, och det hanteras i systemdesignen snarare än i protokollet:

- **Bussbelastningen hålls nere.** Tumregeln i fordonssystem är att inte planera för mer än omkring 30–50 % belastning, just för att lämna utrymme åt lågprioriterade frames.
- **Identifierarna planeras.** Prioritetsordningen sätts av vad som är tidskritiskt, inte av vad som råkade få vilket nummer.
- **Sändningar periodiseras.** En nod som sänder så ofta den kan är en nod som svälter ut alla under sig.

Observera: En frame som förlorat arbitreringen försenas, men blir inte fel. Det är därför CAN kallas *deterministiskt* för den högst prioriterade framen: dess värstafallsfördröjning går att räkna ut. För övriga frames går det också, men svaret blir större ju längre ned i ordningen de ligger.

4.6 Vad sändaren inte får veta

En nod som förlorat arbitreringen vet att den förlorade. En nod som vann vet bara att den kom igenom.

Framför allt: **sändaren får aldrig veta om någon förlorade mot den.** Det finns ingen räknare, ingen rapport, ingen skillnad i framen. Det spelar ingen roll för CAN, men det spelar roll för den som felsöker ett system där en nod aldrig verkar komma fram — symptomet syns bara hos förloraren.

Övningar

Övning 4.1 Två noder

RÄKNING

Nod A sänder identifieraren 0x155, nod B sänder 0x154. Båda börjar samtidigt.

- Skriv upp båda identifierarna binärt, elva bitar, MSB först.
- Vid vilken bit skiljer de sig?
- Vem vinner, och vad gör förloraren?

Övning 4.2 Tre noder

RÄKNING

Tre noder börjar sända samtidigt, med identifierarna 0x400, 0x200 och 0x201.

- Vid vilken bit faller den första noden bort?
- Vid vilken bit avgörs det mellan de två återstående?
- Vem vinner?

Övning 4.3 Samma identifierare

RESONEMANG

Två noder sänder av misstag samma identifierare samtidigt, men med olika data.

- Vad händer under arbitreringsfältet?
- Vad händer när datafälten börjar skilja sig åt?
- Varför är det här ett konstruktionsfel snarare än ett driftfel?

Övning 4.4 Svält

RESONEMANG

En nod med identifieraren 0x7FF kommer aldrig fram på en hårt belastad buss.

- Varför inte?
- Är det ett fel i protokollet? Motivera.
- Ge två åtgärder på systemnivå som skulle hjälpa.

Bittajming

Hur noder utan gemensam klocka läser samma bit samtidigt.

I DET HÄR KAPITLET

- Varför en CAN-bit delas in i segment i stället för att bara räknas.
- Sampelpunkten, och varför den ligger sent i bitperioden.
- Synkronisering: hård vid SOF, mjuk vid varje flank därefter.
- Vad som händer när två noders oscillatorer inte går lika fort.

5.1 Problemet

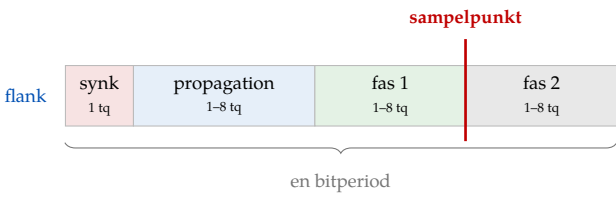
Varje nod har sin egen klocka. Ingen ledning bär takten — CAN har fyra signaler och ingen av dem är en klocka.

Ändå måste alla noder vara överens om var varje bit börjar och slutar, och det på bitnivå: arbitreringen i kapitel 4 bygger på att två noder läser *samma* bitintervall samtidigt och drar slutsatser av vad de ser. Är de ur fas jämför de olika bitar, och resultatet blir godtyckligt.

Lösningen har två delar: en gemensam *definition* av var i bitperioden man läser, och en mekanism som drar tillbaka noder som driver i väg.

5.2 Bitperioden i segment

En CAN-bit är inte ett odelat intervall utan delas in i fyra segment, mätta i **tidskvanta** — korta, lika långa steg som noden räknar med sin egen klocka.



Figur 5.1 Bitperiodens fyra segment. Sampelpunkten ligger mellan fassegment 1 och 2, sent i perioden.

Segment	Längd	Vad det är till för
Synksegment	1 tq	Här förväntas en flank ligga.
Propagationsegment	1–8 tq	Täcker signalens gångtid ut på bussen och tillbaka.
Fassegment 1	1–8 tq	Före sampelpunkten; kan förlängas vid resynkronisering.
Fassegment 2	1–8 tq	Efter sampelpunkten; kan förkortas.

Sampelpunkten ligger mellan fassegment 1 och 2. Det är där, och bara där, biten läses.

5.2.1 Varför sent i perioden

Sampelpunkten läggs typiskt 70–80 % in i bitperioden. Skälet är propagationsegmentet, och det är samma skäl som gör att bussens längd begränsar hastigheten.

Under arbitreringen måste en sändande nod hinna läsa tillbaka vad *den bastersta* noden driver, innan biten är slut. Signalen ska alltså hinna ut och tillbaka. Propagationsegmentet är den tiden, och sampelpunkten måste ligga efter den — annars läser noden bussen innan den andra nodens bit hunnit fram, och arbitreringen blir fel.

Observera: Det är därför bithastighet och kabellängd hänger ihop (kapitel 1). Högre hastighet ger kortare bitperiod, alltså kortare propagationsegment i absolut tid, alltså kortare tillåten kabel. Sambandet är inte elektriskt utan logiskt: det handlar om vad arbitreringen kräver.

5.3 Synkronisering

5.3.1 Hård synkronisering

Vid SOF — den enda flank alla noder ser samtidigt när bussen varit ledig — gör varje mottagare en **hård synkronisering**: den nollställer sin biträknare, oavsett var den stod.

Det är en gång per frame, och det är den enda gången noderna får sätta klockan helt fritt.

5.3.2 Resynkronisering

Under framen räcker inte det. Två oscillatorer som skiljer sig en bråkdel procent driver isär långsamt, och över en frame på 130 bitar blir det mätbart.

Därför resynkroniserar varje mottagare vid *varje* recessiv-till-dominant-flank i strömmen. Ligger flanken tidigare än väntat förkortas fassegment 2; ligger den senare förlängs fassegment 1. Justeringen är begränsad till ett antal tidskvanta som kallas *synchronization jump width*.

Och nu faller bitstopningen på plats. Femregeln i kapitel 3 garanterar att det aldrig går mer än fem bitperioder utan en flank, vilket sätter en övre gräns för hur långt två noder kan hinna driva isär innan de dras tillbaka.

De två mekanismerna är alltså inte oberoende: bitstopningen finns *för* resynkroniseringens skull. Ett protokoll utan stoppning skulle behöva antingen en klockledning eller betydligt noggrannare oscillatorer.

5.4 Att räkna ut tajmingen

För en konstruktion räknas två tal ut ur systemklockan och den önskade bithastigheten:

$$\text{tick per bit} = \frac{f_{\text{system}}}{\text{bithastighet}}$$

Med en systemklocka på 50 MHz och 1 Mbit/s blir det 50 klockcykler per CAN-bit. Sampelpunkten vid 70 % blir då tick 35.

```
// Derived, never written down as a literal: change the bit rate and both follow.
constexpr unsigned TicksPerBit{SystemClockHz / BitRateHz};
constexpr unsigned SampleTick{TicksPerBit * 7U / 10U};
```

Att de *räknas ut* och inte skrivs in är poängen. En konstruktion där 50 och 35 står inskrivna på var sitt ställe är en konstruktion där den ena hinner ändras utan den andra.

5.5 Vad som händer när det går fel

Två fel är värda att känna igen, eftersom de ser olika ut:

Fel bithastighet. Noderna är ur fas från första biten. Symptomet är att ingenting alls fungerar — inga frames tas emot, och den sändande noden rapporterar fel direkt. Det är det snällare felet, för det syns omedelbart.

Fel sampelpunkt. Noderna kommer överens om de flesta bitar men inte alla. Symptomet är sporadiska CRC- och stoppfel som blir värre med längre kabel och högre belastning. Det här är det obehagliga felet, för det ser ut som en dålig kabel.

Övningar

Övning 5.1 Räkna tajmingen

RÄKNING

En konstruktion har systemklockan 50 MHz.

- a) Hur många klockcykler går det på en CAN-bit vid 1 Mbit/s?
- b) Vid 500 kbit/s?
- c) Var ligger sampelpunkten i klockcykler räknat, vid 1 Mbit/s och 70 %?

Övning 5.2 Sampelpunkten

RESONEMANG

- a) Varför ligger sampelpunkten sent i bitperioden i stället för i mitten?
- b) Vad skulle gå fel under arbitreringen om den låg vid 20 %?
- c) Vilken del av bitperioden skulle behöva växa om kabeln blev dubbelt så lång?

Övning 5.3 Stoppning och synk

RESONEMANG

- a) Hur många bitperioder kan som mest passera utan en flank i en CAN-frame?
- b) Vilken mekanism garanterar det?
- c) Vad skulle hända med resynkroniseringen om bitstopppningen togs bort?

Övning 5.4 Två symptom

RESONEMANG

Två noder ska prata med varandra.

- a) Nod A är inställd på 500 kbit/s och nod B på 1 Mbit/s. Vad ser ni?
- b) Båda kör 1 Mbit/s, men A samplar vid 50 % och B vid 80 %. Vad ser ni, och varför är det svårare att felsöka?

Felhantering

Fem sorters fel, en signal, och en nod som stänger av sig själv.

I DET HÄR KAPITLET

- De fem fel en CAN-nod kan upptäcka, och vem som upptäcker vad.
- Felramen: hur en nod avbryter en pågående överföring.
- Felräknarna, och de tre tillstånden en nod kan vara i.
- Bus-off: när en nod kopplar bort sig själv, och varför det är rätt beteende.

6.1 Fem sorters fel

CAN upptäcker fel på fem sätt. Tre av dem har vi redan gått igenom under andra namn:

Fel	Vad det är	Vem ser det
Bitfel	En sändande nod läser tillbaka något annat än den sände	Sändaren
Stoppfel	Sex lika bitar i rad där femregeln kräver en flank	Alla
CRC-fel	Den mottagna checksumman stämmer inte	Mottagare
Formatfel	Ett fält som måste vara recessivt är det inte	Alla
Kvittensfel	Ingen drog ned ACK-luckan	Sändaren

Tillsammans ger de en restfelssannolikhet som brukar anges till omkring 10^{-11} av de fel som uppstår — alltså ungefär ett oupptäckt fel per tusen år på en hårt belastad buss. Det är inte en slump utan resultatet av att fem oberoende mekanismer överlappar.

6.1.1 Bitfel, och undantagen

En sändande nod jämför hela tiden det den sänder med det som ligger på bussen. Skiljer de sig är det ett bitfel — *utom* på två ställen där en skillnad är fullt normal:

- **Under arbitreringsfältet.** Recessiv ut och dominant in betyder förlorad arbitrerings, inte fel (kapitel 4).
- **I ACK-luckan.** Sändaren sänder recessivt och *hoppas* på dominant (kapitel 2).

Att skilja de två fallen från ett riktigt bitfel är en av de saker som gör en CAN-kontroller mer invecklad än den ser ut.

6.2 Felramen

När en nod upptäcker ett fel säger den inte till efteråt. Den avbryter ramen **medan den pågår**, genom att sända en **felframe**: sex dominanta bitar i rad.

Sex dominanta bitar är omöjliga i en korrekt frame — femregeln ser till att det aldrig går mer än fem — så mönstret kan inte förväxlas med data. Varje annan nod på bussen ser det, förstår att ramen är trasig, och kastar den.

Och eftersom alla noder ser felet kommer flera av dem att sända sin egen felflagga i sin tur. Resultatet är en serie dominanta bitar, följd av en avgränsare på åtta recessiva.

Två saker följer:

Alla noder kastar ramen, inte bara den som upptäckte felet. Bussen hålls konsistent: antingen tar alla emot ramen, eller ingen. Det är en starkare garanti än den ett hemmasnickrat protokoll normalt ger sig på.

Sändaren skickar om automatiskt. Kontrollern gör det själv, utan att mjukvaran blir inblandad. En drivrutin som skickar en frame och får veta att det gick bra kan alltså ha fått den omsänd flera gånger utan att märka det.

6.3 Felräknarna

Här kommer den del av CAN som är minst uppenbar och mest genomtänkt.

En nod som råkar ut för fel ska säga ifrån. En nod som är *trasig* ska sluta säga ifrån — annars förstör den bussen för alla andra genom att sända felframes i ett.

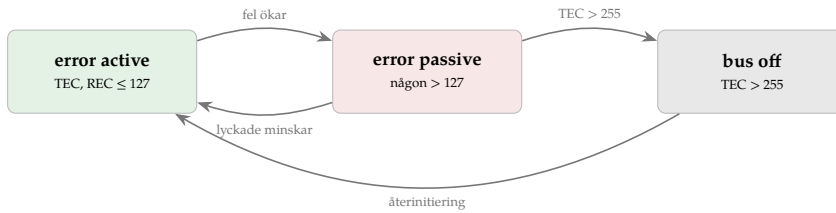
CAN skiljer dem åt med två räknare i varje nod:

- **TEC** — transmit error counter, fel medan noden sände.
- **REC** — receive error counter, fel medan den tog emot.

Reglerna i sammandrag: ett fel ökar räknaren med 8 (sändarsidan) eller 1 (mottagarsidan), och en lyckad överföring minskar den med 1.

Asymmetrin är hela poängen. En nod som verkligen är trasig är inblandad i varje fel och ökar snabbt. En nod som bara råkar vara med på en störig buss ökar långsamt och hinner arbeta av sig. Räknarna mäter alltså inte "hur många fel har hänt" utan "hur sannolikt är det att felet är mitt".

6.3.1 De tre tillstånden



Figur 6.1 En nods tre feltillstånd och övergångarna mellan dem.

Tillstånd	Villkor	Beteende
Error active	TEC och REC ≤ 127	Normal drift; sänder <i>aktiva</i> felflaggor (dominanta)
Error passive	Någon räknare > 127	Sänder <i>passiva</i> felflaggor (recessiva) och väntar extra mellan sändningar
Bus off	TEC > 255	Sänder ingenting alls

Övergången till *error passive* är den intressanta. En passiv felflagga är recessiv, vilket betyder att den inte stör någon annan. Noden får fortfarande delta, den får fortfarande rapportera, men den kan inte längre dra ned bussen för allas räkning.

6.3.2 Bus off

Vid TEC över 255 kopplar noden bort sig själv. Den sänder inte, kvitterar inte, stör inte.

Det är protokollets sista försvar, och det är värt att stanna vid hur ovanligt beteendet är: en nod som konstaterar att den själv troligen är den trasiga delen, och som drar den slutsatsen utan att någon talar om det för den.

En nod i bus off kan tas tillbaka — typiskt efter att ha sett bussen ledig ett visst antal gånger, eller efter att mjukvaran återinitierar kontrollern. Att den återhämtningen finns är också avsiktligt: felet kan ha varit en lös kontakt som nu sitter.

För den som skriver drivrutin: Bus off är det ena tillstånd mjukvaran inte kan ignorera. Allt annat hanterar kontrollern själv — omsändningar, felframes, räknare — men en nod i bus off kommer aldrig tillbaka av sig själv om konstruktionen kräver en återinitiering. Det är ett av de få ställen där felhanteringen når ända upp i applikationen.

6.4 Vad mjukvaran ser

Lägg märke till hur lite av allt det här som når mjukvaran i en fullständig kontroll:

- Fel upptäcks i hårdvara.

- Felframes sänds i hårdvara.
- Omsändning sker i hårdvara.
- Räknarna uppdateras i hårdvara.

Mjukvaran får en statusbit, kanske en felkod, och möjligheten att läsa räknarna. Det är en enorm mängd arbete som gjorts åt den — jämför med ACK, timeout, retry och dubbelthantering i ett hemsnickrat protokoll, som är samma problem löst i mjukvara, sämre.

Förenkling: Den förenklade kontrollern i kapitel 7 implementerar *ingenting* av det här kapitlet utom stoppfelet och CRC-kontrollen. Den sänder inga felframes, har inga felräknare, känner inte till error passive eller bus off, och skickar aldrig om automatiskt. Den signalerar bara att något gick fel, med en enda bit, och lämnar resten till mjukvaran.

Övningar

Övning 6.1 Vem ser felet?

RESONEMANG

För vart och ett av följande: vilken sorts fel är det, och vem upptäcker det?

- En störning flippar en bit i datafältet.
- En störning flippar en bit i ett fält som måste vara recessivt.
- Sändaren är ensam på bussen.
- Sju lika bitar i rad passerar i arbitreringsfältet.

Övning 6.2 Räknarna

RÄKNING

En nod börjar på TEC 0 och råkar ut för åtta sändningsfel i rad, följt av tjugo lyckade sändningar.

- Vad står TEC på efter felen?
- Efter de lyckade sändningarna?
- I vilket tillstånd är noden vid respektive tidpunkt?

Övning 6.3 Asymmetrin

RESONEMANG

Ett fel ökar sändarens räknare med 8 men mottagarens med 1.

- Varför är ökningen olika stor?
- En nod med trasig sändare och en nod på en störig buss ser likadana ut i början. Vad skiljer dem åt efter en stund?

Övning 6.4 Bus off

RESONEMANG

- a) Varför kopplar en nod bort sig själv i stället för att fortsätta rapportera fel?
- b) Vad hade hänt med bussen om den inte gjorde det?
- c) Varför finns en väg tillbaka från bus off?

KAPITEL 7

Den förenklade kontrollern

Vad en undervisningskontroller implementerar, och vad den medvetet låter bli.

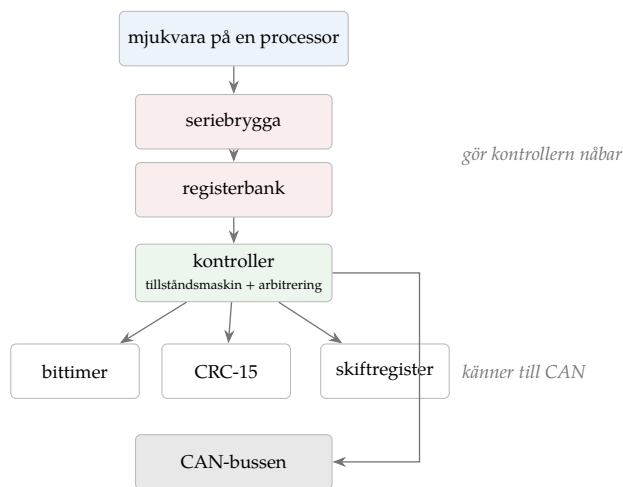
I DET HÄR KAPITLET

- Blocken en förenklad CAN-kontroller består av.
- Vilka delar av kapitel 1–kapitel 6 som finns med, och vilka som inte gör det.
- Varför en pollande mjukvara inte kan prata direkt med kontrollern.
- Registerlagret som gör den nåbar, och de tre begränsningar som når ända upp i koden.

De sex föregående kapitlen beskriver CAN som standarden definierar det. Det här kapitlet beskriver den kontroller som faktiskt byggs i undervisningen: vad den gör av allt det, och vad den låter bli.

Kapitlet vänder sig åt båda hållen. Bygger man kontrollern är listan nedan kravspecifikationen. Skriver man drivrutinen är den i stället listan över vad man inte får räkna med.

7.1 Blocken



Figur 7.1 Den förenklade kontrollerns block, och lagret som gör den nåbar från en processor.

Block	Ansvar
Bittimer	Räknar tick per bit, pulsar vid sampelpunkten och vid bitperiodens slut (kapitel 5).
CRC-15	En bitseriell motor som både beräknar och kontrollerar (kapitel 3).
Sändningsskiftregister	Serialiserar MSB först och sätter in stoppbitar.
Mottagningsskiftregister	Deserialiserar och plockar bort stoppbitar; rapporterar stoppfel.
Kontroller	Tillståndsmaskinen som sekvenserar framens fält och sköter arbitreringen.
Synkroniserare	Två vippor på varje asynkron ingång, så att bussen och reset kan läsas säkert.

Ovanpå dem ligger två block till, som inte har med CAN att göra utan med att göra kontrollern *nåbar*:

Block	Ansvar
Registerbank	Översätter kontrollerns encykelpulser till klibbiga, pollbara register.
Seriebrygga	Gör registren läs- och skrivbara utifrån, över en seriebuss.

7.2 Vad som finns med

Kontrollern implementerar framens sekvens i sin helhet: SOF, arbitreringsfältet med identifierare och RTR, kontrollfältet med IDE, r0 och DLC, datafältet, CRC-15 med avgränsare, ACK-lucka med avgränsare, och sju bitars EOF.

Den implementerar också:

- **Bitstoppning åt båda hållen**, med stoppningstillståndet levande över fältgränser.
- **CRC-15**, med samma motor på sändar- och mottagarsidan.
- **Arbitrering**, bit för bit över arbitreringsfältet.
- **Öppen dränering**: noden driver bara när den ska, och bussen är dominant om någon drar ned den.
- **Stoppfelsdetektering och CRC-kontroll** på mottagarsidan.

Det är tillräckligt för att två sådana noder ska utbyta riktiga CAN-frames på en riktig buss, och tillräckligt för att en kommersiell bussanalysator ska kunna avkoda dem.

7.3 Vad som inte finns med

Saknas	Konsekvens
Felframes	Ett fel avbryter inte överföringen för de andra noderna.
Felräknare	Inget error passive, inget bus off (kapitel 6).
Automatisk omsändning	En frame som gick fel skickas inte om av hårdvaran.
ACK-återläsning	Sändaren får aldrig veta om någon kvitterade.
Mottagning under sändning	En nod som förlorar arbitreringen tar inte emot framen som vann.
Mottagningskö	Kontrollern håller exakt en mottagen frame.
Filtrering	Varje mottagen frame lämnas vidare; filtreringen ligger i mjukvaran.
Utökade identifierare	Bara 11 bitar.
Bitfelsdetektering utanför arbitreringen	Sändaren läser bara tillbaka bussen under arbitreringsfältet.

Var och en av dem är ett medvetet val. Tillsammans är de skillnaden mellan en konstruktion som går att bygga och verifiera under en kurs, och en som inte gör det.

7.4 Varför ett registerlager behövs

Kontrollern är byggd för en anropare i samma klockdomän. Den signalerar med pulser som är **en klockcykel** långa — 20 ns vid 50 MHz.

En processor som pollar över en seriebuss ser systemet några tiotals mikrosekunder i taget i bästa fall. En puls på 20 ns är alltså borta tusentals cykler innan nästa pollning kommer.

Registerbanken löser det genom att göra om pulser till nivåer. En puls sätter en bit, och biten står kvar tills mjukvaran uttryckligen nollställer den:

Statusbit	Sätts av	Nollställs av
TX klar	en avslutad sändning	en accepterad sändningstrigger
RX giltig	en mottagen frame	en kvittering från mjukvaran
Fel	en stigande flank på felnivån	en uttrycklig nollställning

Det mönstret — *klibbig bit, uttrycklig nollställning* — är hela poängen med lagret, och det är också den vanligaste källan till buggar i mjukvaran ovanför. Glöms nollställningen står biten kvar för alltid, och drivrutinen läser samma frame i en oändlig loop.

Observera: Registerbanken *döljer* inte kontrollerns begränsningar, den *vidarebefordrar* dem. Det är ett designval värt att lägga märke till: ett lager som hade buffrat mottagna frames hade sett hjälpsamt ut och gjort systemets verkliga beteende omöjligt att resonera om.

7.5 De tre begränsningarna som når ända upp

Tre av förenklarna ovan är inte bara interna. De syns i mjukvaran, och den som skriver drivrutinen måste hantera dem snarare än att bli förvånad av dem.

7.5.1 En förlorad arbitrering ser ut som en lyckad sändning

Kontrollern signalerar "sändningen är över" i båda fallen. Biten betyder **porten är fri igen**, inte *ramen kom fram*.

Att den gör så är avsiktligt, och det är faktiskt en rättelse av en verklig bugg: signalerade den bara vid lyckad sändning, skulle biten bli liggande efter varje förlorad arbitrering, och noden kunde inte sända igen förrän den nollställdes för hand. På en tvånodsbus alltså i normal drift.

Konsekvensen för mjukvaran är en pollningsloop i två steg:

```
vänta på "porten är fri"  -> sändningsförsöket är över
läs felflaggan           -> gick det bra eller inte?
```

Utan det andra steget går det inte att veta om ramen ska skickas om.

7.5.2 En bit för fyra orsaker

Förlorad arbitrering, stoppfel, misslyckad CRC och mottagen fjärrframe ger alla samma enda felflagga. Mjukvaran kan se *att* något gick fel, aldrig *vad*.

Retry-logik som behandlar en förlorad arbitrering annorlunda än en trasig CRC går alltså inte att skriva mot det här gränssnittet. Rätt hantering är att veta om begränsningen och behandla alla fyra lika — inte att gissa orsaken ur sammanhanget.

7.5.3 Ingen buffring på mottagarsidan

Kontrollern håller en frame. Kommer nästa innan den förra kvitterats skrivs den över, och den nyaste vinner.

Det är dokumenterat och avsiktligt. En riktig kontroller har en mottagningskö; den här har det inte, och mjukvaran får polla tillräckligt ofta. Vid någon trafikintensitet räcker det

inte oavsett hur ofta den pollar, och då tappas frames — vilket är hårdvarans begränsning och inte mjukvarans bugg.

7.6 Vad en riktig kontroller skulle lägga till

För fullständighetens skull, och som svar på frågan ”hur långt är det kvar?”:

- Felframes, felräknare och bus off (kapitel 6).
- Automatisk omsändning efter fel.
- Mottagningsfilter i hårdvara, så att ointressanta frames aldrig når mjukvaran.
- En mottagningskö med flera platser.
- Flera sändningsbuffertar med inbördes prioritet.
- Utökade identifierare, och därmed CAN 2.0B.
- Återläsning av hela ramen, och därmed riktig bitfelsdetektering.

Listan är lång, men lägg märke till vad som *inte* står på den: ramning, stoppning, CRC, arbitrering och bittajming. Det svåra i CAN är redan gjort.

Övningar

Övning 7.1 Vad saknas

RESONEMANG

För vart och ett av följande scenarier: vad gör en fullständig CAN-kontroller, och vad gör den förenklade?

- En frame förlorar arbitreringen.
- En mottagen frame har fel checksumma.
- Ingen nod kvitterar en sänd frame.
- Två frames tas emot tätt efter varandra.

Övning 7.2 Klibbiga bitar

RESONEMANG

Kontrollern pulsar i en klockcykel; mjukvaran pollar var fjortionde mikrosekund.

- Hur många klockcykler på 50 MHz går det mellan två pollningar?
- Vad skulle mjukvaran se om statusbiten speglade pulsen direkt?
- Varför måste biten nollställas av en skrivning i stället för automatiskt vid läsning?

Övning 7.3 Två steg

RESONEMANG

En drivrutin väntar på att sändningen ska bli klar och returnerar sedan "det gick bra".

- a) Vilket fel gör den?
- b) Vad måste den göra i stället?
- c) Vilka fyra orsaker kan ligga bakom en satt felflagga?

Övning 7.4 Gränsen

RESONEMANG

Registerbanken vidarebefordrar kontrollernas begränsningar i stället för att dölja dem.

- a) Ge ett exempel på hur den *skulle* kunna dölja en av dem.
- b) Vad hade det kostat den som felsöker systemet?
- c) Vilken princip säger detta något om, mer allmänt, när man konstruerar ett gränssnitt mellan hårdvara och mjukvara?

Svar på övningarna

Här finns svaren på samtliga övningar. Läs dem efter att ni försökt själva — ett svar man läst är inte samma sak som ett svar man kommit fram till, och det är den skillnaden övningarna finns för.

Där ett svar bygger på en uträkning står uträkningen med, inte bara resultatet.

A.1 Bussen

Övning 1.1 Kablage

sidan 5

- a) $8 \cdot 7/2 = 28$ förbindelser.
- b) 8 anslutningar, en per enhet.
- c) Vi söker n där $n(n-1)/2 = 2n$, alltså $n-1 = 4$ och $n = 5$. Redan vid fem enheter är punkt-till-punkt dubbelt så mycket koppar, och avståndet växer sedan snabbt.

Övning 1.2 Bussens nivå

sidan 5

- a) **dominant**. Bussen är dominant så snart minst en nod drar ned den, och nod B gör det.
- b) Nod A och nod C. Båda sände recessivt och läser tillbaka dominant. Nod D sände ingenting och har inget att jämföra med.
- c) Att någon annan sänder samtidigt, och att den andra har företräde. Sker det i arbitreringsfältet har A och C förlorat arbitreringsfältet och blir mottagare.

Övning 1.3 Terminering

sidan 5

- a) Det sitter tre termineringsmotstånd i stället för två: tre parallella $120\ \Omega$ ger $40\ \Omega$. Troligen har en nod mitt på bussen ett motstånd påslaget.
- b) För låg terminering dämpar signalen men förstör den inte på en kort kabel vid måttlig hastighet — flankerna hinner ändå ställa sig. Felet visar sig när kabeln blir längre, farten högre eller miljön störigare, alltså typiskt när konstruktionen flyttas från labbänken.

Övning 1.4 Asymmetrin

sidan 5

- a) Ja. Arbitreringen kräver bara att *ett* av de två tillstånden vinner entydigt; vilket spelar ingen roll för mekanismen.
- b) Högst identifierare skulle vinna, eftersom en etta då skrev över en nolla.
- c) Tänk på vad bussen gör när ingen sänder: den vilar i det tillstånd ingen driver, alltså recessivt. Med den valda ordningen är viloläget det *förlorande* tillståndet, vilket betyder att en nod som slutar sända automatiskt drar sig ur. Vore det tvärtom skulle en tyst buss se ut som en nod som vinner allt, och en nod som tappade strömmen skulle blockera bussen.

A.2 Framen**Övning 2.1 Läs framen**

sidan 11

- a) $1 + 12 + 6 + 40 + 16 + 2 + 7 = 84$ bitar. SOF 1, arbitreringsfält 12 (11 identifierare + RTR), kontrollfält 6, data $5 \cdot 8 = 40$, CRC-fält 16 (15 + avgränsare), ACK-fält 2, EOF 7.
- b) $84\ \mu\text{s}$ vid 1 Mbit/s, före stoppning. Med stoppbitar och de tre bitarna intermission blir det i praktiken något mer.
- c) $0 \times 2A6$, eftersom lägre numeriskt värde vinner.

Övning 2.2 Ingen adress

sidan 11

- a) En. Sensorn sänder temperaturen en gång, med en identifierare som beskriver *innehållet*.
- b) Två frames, en till vardera mottagaren, eller en broadcast-adress som protokollet måste definiera särskilt.
- c) Ingenting. Den tredje noden börjar bara lyssna efter samma identifierare. Det är CAN:s starkaste argument: mottagare kan läggas till utan att sändaren vet om det.

Övning 2.3 Kvittensen

sidan 12

- a) Att minst en nod tog emot framen med korrekt CRC.
- b) Hur många som gjorde det, vilka de var, om någon av dem brydde sig om innehållet, och om någon faktiskt behandlade den. Kvittensen gäller överföringen, inte behandlingen.
- c) Den läser recessivt, eftersom ingen annan finns som kan dra ned luckan. En standardenlig kontroller tolkar det som ett kvittensfel, ökar sin sändarfelräknare och skickar om — vilket är varför en ensam nod på en CAN-buss sänder samma frame om och om igen tills någon svarar.

Övning 2.4 DLC

sidan 12

- a) Fältet ligger i kontrollfältet vars bredd var bestämd, och fyra bitar var vad som fanns. Att bara 0–8 används är priset för ett fält som är jämnt i bitantal snarare än snålt.
- b) Drivrutinens. Registerlagret vidarebefordrar värdet som det är, och kontrollern sekvenserar efter det — ingen av dem validerar. Att lägga kontrollen på båda ställena är precis så de två kopiorna av regeln glider isär; den hör hemma där framen skapas.

A.3 Bitstoppning och CRC-15**Övning 3.1 Stoppa en ström**

sidan 16

- a) 0000011111 blir 000001111101: en stoppbit efter de fem nollorna och en efter de fem ettorna. 10 bitar blir 12.
- b) 1111111111 blir 111110111110: en stoppbit efter de första fem ettorna, och eftersom stoppbiten själv bryter serien räknas de följande fem ettorna om från början, vilket ger ytterligare en. 10 bitar blir 12.
- c) 0101010101 är oförändrad. Ingen serie når fem, så ingen stoppbit behövs. 10 bitar förblir 10.

Övning 3.2 Avstoppa

sidan 16

- a) Biten på position 6 (nollan efter fem ettor) och biten på position 11 (ettan efter fem nollor). Positionerna räknas från ett.
- b) 1111100000.
- c) Ett stoppfel, efter den sjätte biten. Efter fem lika bitar *måste* nästa vara den motsatta; är den det inte är bitströmmen trasig, och mottagaren förkastar framen.

Övning 3.3 CRC-motorn

sidan 17

Registret efter varje bit, med start från noll:

Bit	Värde	Registret efter	Hex
1	1	100010110011001	0x4599
2	1	000101100110010	0x0B32
3	0	001011001100100	0x1664
4	1	110100101010001	0x6951

Lägg märke till steg 2 och 3: när registrets översta bit och den inkommande biten är lika blir återkopplingen noll, och steget är en ren skiftning utan XOR.

Övning 3.4 Varför noll?

sidan 17

- a) Checksumman är resten vid division med polynomet. Att lägga resten sist i dividenden gör den jämnt delbar, och en jämn division lämnar resten noll. Mottagaren behöver därför ingen jämförelse — den tittar bara efter om registret är nollställt.
- b) Samma modul kan användas på både sändar- och mottagarsidan, utan lägesomkoppling och utan en separat jämförelsekreter. I hårdvara är det en påtaglig besparing, och en felkälla mindre.
- c) Sändarens och mottagarens checksummor beräknas då över olika bitströmmar, eftersom stoppbitarna beror på datan. Resultatet blir att varje frame som innehåller minst en stoppbit förkastas som CRC-fel — alltså nästan alla. Symptomet är förvirrande: korta frames med växlande data fungerar, längre frames gör det inte.

A.4 Arbitrering

Övning 4.1 Två noder

sidan 22

- a) 0x155 är 00101010101 och 0x154 är 00101010100.
- b) Vid bit 11, den sista biten i identifieraren.
- c) B vinner, eftersom den sänder dominant där A sänder recessivt. A upptäcker det omedelbart, slutar sända, blir mottagare för B:s frame och försöker igen när bussen blivit ledig.

Övning 4.2 Tre noder

sidan 22

Binärt: 0x400 är 100000000000, 0x200 är 010000000000 och 0x201 är 010000000001.

- a) Vid bit 1. A sänder recessivt medan B och C sänder dominant, så A faller bort direkt.
- b) Vid bit 11. B och C är identiska i tio bitar och skiljer sig först i den sista.
- c) B, med identifieraren 0x200.

Övning 4.3 Samma identifierare

sidan 22

- a) Ingenting. Bitarna är identiska hela arbitreringsfältet igenom, så ingen av noderna ser någon skillnad mellan sig själv och bussen. Båda tror att de vann.
- b) Så fort datafälten skiljer sig sänder den ena noden recessivt och läser dominant. Utanför arbitreringsfältet är det inte en förlorad arbitrering utan ett *bitfel*: noden avbryter med en felframe, båda frames förstörs, och båda noderna skickar om — varpå samma sak händer igen.
- c) Därför att det inte kan uppstå i drift av en korrekt konfigurerad buss. Identifierare tilldelas per sändande nod vid systemdesignen; två noder med samma identifierare är ett fel i tilldelningen, inte något som händer av sig självt.

Övning 4.4 Svält

sidan 22

- a) Den förlorar varje arbitrering mot varje annan nod, eftersom 0x7FF är det högsta möjliga värdet och därmed lägst prioritet. Är bussen alltid upptagen kommer den aldrig fram.
- b) Nej. Protokollet garanterar att den *högst* prioriterade ramen kommer fram inom en förutsägbar tid, och det är vad determinism i CAN betyder. Att lågprioriterad trafik kan svälta är en konsekvens av prioritetsordningen, och den ordningen är poängen.
- c) Till exempel: hålla bussbelastningen nere (30–50 % är tumregeln i fordonssystem); planera identifierarna efter vad som är tidskritiskt; periodisera sändningarna så att ingen nod sänder så ofta den kan; eller dela upp trafiken på flera bussar.

A.5 Bittajming

Övning 5.1 Räkna tajmingen

sidan 26

- a) $50\,000\,000 / 1\,000\,000 = 50$ klockcykler per bit.
- b) $50\,000\,000 / 500\,000 = 100$ klockcykler per bit.
- c) $50 \cdot 0,7 = 35$, alltså vid tick 35 av 50.

Övning 5.2 Sampelpunkten

sidan 26

- a) För att signalen ska hinna ut till bussens bitersta nod och tillbaka innan biten läses. Det är propagationssegmentets uppgift, och sampelpunkten måste ligga efter det.
- b) Under arbitreringen skulle en nod läsa bussen innan en annan nods dominanta bit hunnit fram, och alltså tro att den vann när den förlorade. Två noder skulle då sända samtidigt resten av framen.
- c) Propagationssegmentet. Det är det segment som representerar gångtiden, och det är därför kabellängd och bithastighet hänger ihop.

Övning 5.3 Stoppning och synk

sidan 26

- a) Fem.
- b) Bitstopppningen, femregeln. Efter fem lika bitar sätts alltid en bit med motsatt värde in, vilket garanterar en flank.
- c) Mottagarna skulle kunna gå långa sträckor utan flanker att resynkronisera mot, och driva i väg tills de samplar fel bit. Det skulle i praktiken kräva en klockledning eller betydligt noggrannare oscillatorer — vilket är själva skälet till att stopppningen finns.

Övning 5.4 Två symptom

sidan 26

- a) Ingenting fungerar. Noderna är ur fas redan från SOF, så varje frame blir obegriplig i andra änden. Felet syns omedelbart och är därför det snällare av de två.
- b) De flesta bitar läses lika, men de som ligger nära en flank eller påverkas av propagationsfördröjning läses olika. Symptomet blir sporadiska CRC- och stoppfel som blir värre med längre kabel och högre belastning. Det är svårare att felsöka därför att det ser ut som en dålig kabel eller en störig miljö, och därför att det fungerar tillräckligt bra för att man ska tro att grundinställningen är rätt.

A.6 Felhantering

Övning 6.1 Vem ser felet?

sidan 30

- a) CRC-fel. Mottagarna upptäcker det; sändaren får veta det indirekt, genom att ingen kvitterar.
- b) Formatfel. Alla noder ser det, eftersom alla vet vilka bitar som måste vara recessiva.
- c) Kvittensfel. Bara sändaren ser det: den släpper ACK-luckan och läser tillbaka recessivt.
- d) Stoppfel. Alla noder ser det, eftersom alla räknar lika bitar i rad.

Övning 6.2 Räknarna

sidan 30

- a) $8 \cdot 8 = 64$.
- b) $64 - 20 = 44$.
- c) Error active i båda fallen, eftersom gränsen till error passive går vid 127.
Lägg märke till proportionerna: åtta fel kostar 64, och det krävs 64 lyckade sändningar för att arbeta av dem. Räknaren är medvetet trögare på vägen ned än på vägen upp.

Övning 6.3 Asymmetrin

sidan 30

- a) Därför att räknarna inte mäter hur många fel som inträffat, utan hur sannolikt det är att felet är nodens eget. En sändande nod är inblandad i varje fel den ser; en mottagande nod är en av många som såg samma fel. Den som sänder är alltså en mer trolig felkälla, och straffas därefter.
- b) Den trasiga noden är inblandad i *varje* fel och klättrar åtta steg i taget, medan den friska noden på en störig buss ökar ett steg i taget och hinner arbeta av sig mellan störningarna. Efter en stund är den trasiga noden error passive eller bus off medan den friska fortfarande är error active.

Övning 6.4 Bus off

sidan 31

- a) Därför att en nod med en trasig sändare rapporterar fel på varje frame — också på de frames som är helt korrekta. Dess felframes är dominanta och förstör andras trafik, så en enda trasig nod kan lägga hela bussen.
- b) Den hade blivit oanvändbar. Varje frame från varje nod hade avbrutits av den trasiga nodens felflagga, och all trafik hade upphört.
- c) Därför att felet kan vara övergående: en lös kontakt, en störning, en spänningsdipp. En permanent avstängning hade krävt att någon åker ut och startar om noden för hand, också när felet sedan länge är borta.

A.7 Den förenklade kontrollern**Övning 7.1 Vad saknas**

sidan 37

- a) Fullständig: förloraren blir mottagare och tar emot den vinnande framen, och skickar om sin egen automatiskt när bussen blivit ledig. Förenklad: förloraren sätter sin felflagga, lämnar sändarrollen och går till vila — den tar inte emot framen som vann, och skickar inte om.
- b) Fullständig: mottagaren avbryter med en felframe, alla noder kastar framen, och sändaren skickar om. Förenklad: mottagaren förkastar framen och sätter felflaggan. Ingen felframe sänds, så sändaren vet ingenting.
- c) Fullständig: sändaren upptäcker kvittensfelet och skickar om. Förenklad: sändaren läser aldrig tillbaka ACK-luckan, så överföringen fullbordas precis som om allt gått bra.
- d) Fullständig: båda hamnar i mottagningskön. Förenklad: den andra skriver över den första om den första inte hunnit kvitteras, och den nyaste vinner.

Övning 7.2 Klibbiga bitar

sidan 37

- a) $40 \cdot 10^{-6} \cdot 50 \cdot 10^6 = 2000$ klockcykler.
- b) Så gott som ingenting. Pulsen är en cykel lång och sannolikheten att en pollning råkar träffa just den cykeln är ungefär 1 på 2000. Mjukvaran skulle missa nästan varje händelse, och de få den fångade skulle se slumpmässiga ut.
- c) Därför att en automatisk nollställning vid läsning gör det omöjligt att läsa statusen två gånger, och därmed omöjligt att skriva kod som först kontrollerar en bit och sedan agerar på den. En uttrycklig nollställning skiljer dessutom "jag har sett händelsen" från "jag har hanterat den" — och det är mjukvaran, inte hårdvaran, som vet när det senare är sant.

Övning 7.3 Två steg

sidan 38

- a) Den förväxlar *porten är fri igen* med *ramen kom fram*. Statusbiten kommer tillbaka både efter en lyckad sändning och efter en förlorad arbitrerings.
- b) Vänta på att porten blir fri, och sedan läsa felflaggan. Först den kombinationen säger hur det gick.
- c) Förlorad arbitrerings, stoppfel, misslyckad CRC-kontroll, och en mottagen fjärrframe. Alla fyra ger samma enda bit, så drivrutinen kan inte skilja dem åt.

Övning 7.4 Gränsen

sidan 38

- a) Till exempel genom att buffra mottagna frames i en kö, så att mjukvaran aldrig märkte att kontrollern bara håller en. Eller genom att automatiskt nollställa statusbiten vid läsning, så att mjukvaran slapp kvittera.
- b) Ett dolt beteende är ett beteende man inte kan resonera om. En kö som ibland svämmar över ger frames som försvinner utan spår, och felsökningen börjar på fel ställe: i mjukvaran, som gjorde rätt. En begränsning man känner till går att planera för; en man inte känner till går bara att stöta på.
- c) Att ett gränssnitt ska vara *ärligt* snarare än hjälpsamt. Ett lager som döljer den underliggande hårdvarans egenskaper flyttar inte problemet, det gör det bara osynligt — och det som är osynligt går inte att testa, mäta eller resonera om. Samma princip gäller ett drivrutins-API: det ska säga vad hårdvaran gör, inte vad man önskar att den gjorde.